

Bash Arrays A to Z

Great—this is exactly the kind of skill that makes a strong Red Hat Enterprise Linux admin stand out. Bash arrays are *practical*, not just theory—you'll use them in automation, scripting, patching, and monitoring.

Let's go **A → Z**, simple but complete.

1. What is an Array in Bash?

An array = a variable that holds **multiple values**.

```
</> Bash
servers=("web1" "web2" "db1")
```

Instead of:

```
</> Bash
server1="web1"
server2="web2"
```

2. Types of Arrays in Bash

Indexed Array (most common)

```
</> Bash
arr=("a" "b" "c")
```

Indexes:

```
0 → a
1 → b
2 → c
```

Associative Array (key-value)

</> Bash

```
declare -A user_role
user_role[admin]="root"
user_role[dev]="appuser"
```



3. How to Create Arrays

Method 1: Direct

</> Bash

```
names=("john" "doe" "alice")
```

Method 2: Index assignment

</> Bash

```
names[0]="john"
names[1]="doe"
```

Method 3: From command output (VERY useful in sysadmin work)

</> Bash

```
files=$(ls /etc)
```



4. Accessing Array Elements

</> Bash

```
echo ${names[0]} # john
echo ${names[1]} # doe
```

All elements:

</> Bash

```
echo ${names[@]}
```

5. Loop Through Arrays (MOST IMPORTANT)

Best practice loop:

```
</> Bash
for name in "${names[@]}"
do
    echo "User: $name"
done
```

👉 You'll use this for:

- looping servers
- looping users
- looping files

6. Array Length

```
</> Bash
echo ${#names[@]}
```

👉 Useful for checks:

```
</> Bash
if [ ${#names[@]} -eq 0 ]; then
    echo "Empty array"
fi
```

7. Add Elements

```
</> Bash
names+=("newuser")
```

8. Remove Elements

`</> Bash`

```
unset names[1]
```

9. Slice Arrays

`</> Bash`

```
echo ${names[@]:1:2}
```

 Meaning:

- start index = 1
- take 2 elements

10. Real Sysadmin Use Cases

Example 1: Loop servers and ping

`</> Bash`

```
servers=("web1" "web2" "db1")

for server in "${servers[@]}"
do
    ping -c 1 $server
done
```

Example 2: Create users

`</> Bash`

```
users=("dev1" "dev2" "dev3")

for user in "${users[@]}"
do
    useradd $user
done
```

Example 3: Process files

```
</> Bash
files=$(ls /var/log)

for file in "${files[@]}"
do
    echo "Processing $file"
done
```

11. Associative Arrays (IMPORTANT for configs)

```
</> Bash
declare -A ports
ports[http]=80
ports[ssh]=22

echo ${ports[ssh]}
```

Loop:

```
</> Bash
for key in "${!ports[@]}"
do
    echo "$key -> ${ports[$key]}"
done
```

12. Common Mistakes (VERY IMPORTANT)

Not quoting arrays

```
</> Bash
for i in ${arr[@]} # BAD
```

Correct

```
</> Bash
```

```
for i in "${arr[@]}"
```

✗ Using `ls` in scripts (risky)

Better:

```
</> Bash
files=(/var/log/*)
```

13. Debugging Arrays

```
</> Bash
set -x
```

Print array clearly:

```
</> Bash
declare -p names
```

14. When You MUST Use Arrays

As a sysadmin, use arrays for:

- Multiple servers
- Batch operations
- Config mappings
- Automation loops
- Parsing command output

15. Mini Real Project (Practice)

Script: Check multiple services

</> Bash

```
services=("sshd" "nginx" "docker")

for service in "${services[@]}"
do
    systemctl is-active --quiet $service
    if [ $? -eq 0 ]; then
        echo "$service is running"
    else
        echo "$service is NOT running"
    fi
done
```

16. What You Should Master (Checklist)

- ✓ Creating arrays
- ✓ Accessing values
- ✓ Looping arrays
- ✓ Adding/removing elements
- ✓ Array length
- ✓ Associative arrays
- ✓ Using arrays with commands
- ✓ Writing automation scripts

17. Learning Path (Recommended)

1. Basics (create, access)
2. Looping
3. Real sysadmin scripts
4. Associative arrays
5. Error handling + debugging
6. Combine with:
 - `grep`
 - `awk`
 - `sed`

💡 Final Advice (From Real Sysadmin Perspective)

Don't just memorize—use arrays in scripts daily:

- Patch multiple servers
- Monitor services
- Automate backups
- User provisioning

That's how you become strong.

If you want, I can:

- 👉 Give you **real interview questions** (Red Hat level)
- 👉 Give **hands-on labs** like RHCSA/RHCE style
- 👉 Or review your scripts and improve them

Just tell me 👍

Perfect—this is where Bash arrays become **real power tools** for a system administrator (especially in environments like Red Hat Enterprise Linux).

I'll go beyond basics and show you **how arrays are actually used in production**, with **practical scenarios + scripts + why they matter**.

🕒 1. Managing Multiple Servers (VERY COMMON)

🎯 Scenario

You manage 20+ servers and want to run the same command everywhere.

✅ Solution using arrays

```
<> Bash
servers=("web1" "web2" "db1" "db2")

for server in "${servers[@]}"
do
    echo "Checking $server..."
```

```
ssh $server "uptime"
done
```

💡 Why arrays?

- Central list of servers
- Easy to update
- Reusable across scripts

2. Batch Service Management

Scenario

Check multiple services status on a server.

```
</> Bash
services=("sshd" "nginx" "docker" "cron")

for svc in "${services[@]}"
do
    if systemctl is-active --quiet $svc; then
        echo "$svc is running"
    else
        echo "$svc is NOT running"
    fi
done
```

💡 Real-world use:

- Health checks
- Monitoring scripts
- Cron jobs

3. Bulk User Management

Scenario

Create multiple users at once.

```
</> Bash
```

```
users=("dev1" "dev2" "qa1")

for user in "${users[@]}"
do
    id "$user" &>/dev/null
    if [ $? -ne 0 ]; then
        useradd "$user"
        echo "Created $user"
    else
        echo "$user already exists"
    fi
done
```

💡 Why important?

- Onboarding automation
- Avoid manual repetitive work

📁 4. File and Log Processing

🎯 Scenario

Process log files daily.

```
<> Bash

logs=(/var/log/*.log)

for log in "${logs[@]}"
do
    echo "Checking $log"
    grep "ERROR" "$log" >> errors.txt
done
```

💡 Real-world:

- Log analysis
- Security monitoring
- Incident response

🌐 5. Port / Service Mapping (Associative Arrays)

Scenario

Map service names to ports.

```
</> Bash
declare -A ports
ports[http]=80
ports[https]=443
ports[ssh]=22

for service in "${!ports[@]}"
do
    echo "$service runs on port ${ports[$service]}"
done
```

Why associative arrays?

- Clean config-style scripting
- Easy lookup

6. Backup Automation

Scenario

Backup multiple directories.

```
</> Bash
dirs=("/etc" "/home" "/var/www")

for dir in "${dirs[@]}"
do
    tar -czf "/backup$(basename $dir).tar.gz" "$dir"
    echo "Backed up $dir"
done
```

7. Monitoring Disk Usage

Scenario

Check important mount points.

</> Bash

```
mounts=("/" "/home" "/var")

for mnt in "${mounts[@]}"
do
    usage=$(df -h "$mnt" | awk 'NR==2 {print $5}')
    echo "$mnt usage: $usage"
done
```



8. Running Commands Conditionally



Scenario

Restart only failed services.

</> Bash

```
services=("nginx" "docker" "sshd")

for svc in "${services[@]}"
do
    if ! systemctl is-active --quiet $svc; then
        echo "Restarting $svc..."
        systemctl restart $svc
    fi
done
```



9. Package Installation (Automation)



Scenario

Install required packages on a new server.

</> Bash

```
packages=("vim" "git" "curl" "htop")

for pkg in "${packages[@]}"
do
    yum install -y "$pkg"
done
```

👉 Or modern:

```
</> Bash
dnf install -y "${packages[@]}"
```

10. Combining Arrays with Command Output

Scenario

Get running processes and act on them.

```
</> Bash
pids=$(pgrep nginx)

for pid in "${pids[@]}"
do
    echo "NGINX running with PID: $pid"
done
```

11. Advanced: Parallel Operations (Basic Idea)

```
</> Bash
servers=("web1" "web2" "web3")

for server in "${servers[@]}"
do
    ssh "$server" "uptime" &
done

wait
```

Why?

- Faster execution
- Used in large environments

12. Real Admin Pitfalls (Learn This!)

✗ Spaces breaking scripts

```
</> Bash
files=(/home/*)
```

Always quote:

```
</> Bash
for f in "${files[@]}"
```

✗ Unsafe SSH loops

Always handle errors:

```
</> Bash
ssh "$server" "uptime" || echo "Failed: $server"
```



13. Real-Life Mini Project (Admin Script)



Health Check Script

```
</> Bash
#!/bin/bash

servers=("web1" "db1")
services=("nginx" "sshd")

for server in "${servers[@]}"
do
    echo "===== $server ====="

    for svc in "${services[@]}"
    do
        ssh "$server" "systemctl is-active --quiet $svc"

        if [ $? -eq 0 ]; then
            echo "$svc OK"
        else
            echo "$svc FAILED"
        fi
    done
done
```

fi
done
done

14. What You Should Focus on as a Sysadmin

Must-master skills:

- Arrays + loops together
- Arrays with SSH
- Arrays with systemctl
- Arrays with file paths
- Associative arrays for configs

Final System Admin Advice

Arrays = automation multiplier

Without arrays:

-  manual work
-  repetitive commands
-  errors

With arrays:

-  scalable scripts
-  clean automation
-  production-ready tools

If you want next level:

-  I can give RHCSA/RHCE-style labs
-  Real interview questions + answers
-  Build a complete admin toolkit script pack

Just tell me 

Do you like this personality?